

```
!apt-get update -qq
```

```
!apt-get install openjdk-11-jdk -qq
```

```
!java -version
```

```
!wget https://archive.apache.org/dist/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
```

```
!tar -xzf hadoop-3.3.6.tar.gz
```

```
!!ls
```

```
!mv hadoop-3.3.6 /usr/local/hadoop
```

```
!!ls /usr/local/hadoop
```

```
import os os.environ["HADOOP_HOME"]="/usr/local/hadoop"
```

```
os.environ["JAVA_HOME"]="/usr/lib/jvm/java-11-openjdk-amd64"
```

```
os.environ["PATH"] += ":/usr/local/hadoop/bin"
```

```
os.environ["PATH"] += ":/usr/local/hadoop/sbin"
```

```
!/usr/local/hadoop/bin/hadoop version
```

```
!!ls
```

```
!hadoop version
```

```
!find / -name core-site.xml 2>/dev/null
```

```
!sed -i 's|<configuration>|\
```

```
<property>\
```

```
<name>fs.defaultFS</name>\
```

```
<value>hdfs://localhost:9000</value>\
```

```
</property>\
```

```
</configuration>|' /usr/local/hadoop/etc/hadoop/core-site.xml
```

```
!sed -i 's|<configuration>|\
```

```
<property>\
<name>dfs.replication</name>\
<value>1</value>\
</property>\
</configuration>|' /usr/local/hadoop/etc/hadoop/hdfs-site.xml
```

```
!hdfs namenode -format
```

```
!echo 'export HDFS_NAMENODE_USER=root' >> ~/.bashrc
!echo 'export HDFS_DATANODE_USER=root' >> ~/.bashrc
!echo 'export HDFS_SECONDARYNAMENODE_USER=root' >> ~/.bashrc
!echo 'export YARN_RESOURCEMANAGER_USER=root' >> ~/.bashrc
!echo 'export YARN_NODEMANAGER_USER=root' >> ~/.bashrc
!source ~/.bashrc
```

```
%%writefile /usr/local/hadoop/etc/hadoop/core-site.xml
```

```
<?xml version="1.0"?>
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

```
!export HDFS_NAMENODE_USER=root
!export HDFS_DATANODE_USER=root
!export HDFS_SECONDARYNAMENODE_USER=root
!export YARN_RESOURCEMANAGER_USER=root
!export YARN_NODEMANAGER_USER=root
```

```
!echo $HDFS_NAMENODE_USER
```

```
%%writefile /usr/local/hadoop/etc/hadoop/hdfs-site.xml
```

```
<?xml version="1.0"?>
<configuration>
  <property>
```

```

    <name>dfs.replication</name>

    <value>1</value>
</property>
<property>

    <name>dfs.namenode.name.dir</name>

    <value>file:///usr/local/hadoop/hdfs/namenode</value>
</property>
<property>

    <name>dfs.datanode.data.dir</name>

    <value>file:///usr/local/hadoop/hdfs/datanode</value>
</property>
</configuration>

```

```
!mkdir -p /usr/local/hadoop/hdfs/namenode
```

```
!mkdir -p /usr/local/hadoop/hdfs/datanode
```

```
!sed -i '$a export HDFS_NAMENODE_USER=root' /usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

```
!sed -i '$a export HDFS_DATANODE_USER=root' /usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

```
!sed -i '$a export HDFS_SECONDARYNAMENODE_USER=root' /usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

```
!tail -n 5 /usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

```
!java -version
```

```
!apt-get update -qq
```

```
!apt-get install openjdk-8-jdk -qq
```

```
!readlink -f /usr/bin/java | sed "s:bin/java::"
```

```
!sed -i 's|^export JAVA_HOME=.*|export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64|'
/usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

```
!echo "export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64" >> /usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

```
!grep JAVA_HOME /usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

```
!export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
```

```
!jps          !ls
```

Step 1: Import Libraries

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler
```

Step 2: Load IPL CSV Files

```
matches = pd.read_csv("ipl_matches_data.csv")
```

```
balls = pd.read_csv("ball_by_ball_data.csv")
```

Step 3: Create Match Statistics

```
runs_per_match = balls.groupby("match_id")["total_runs"].sum().reset_index()
```

```
wickets_per_match = balls.groupby("match_id")["is_wicket"].sum().reset_index()
```

Calculate total extra runs per ball first, then per match

```
balls["calculated_extra_runs"] = balls["wide_ball_runs"] + balls["no_ball_runs"] + \
```

```
    balls["leg_bye_runs"] + balls["bye_runs"] + \
```

```
    balls["penalty_runs"]
```

```
extras_per_match = balls.groupby("match_id")["calculated_extra_runs"].sum().reset_index()
```

Merge datasets

```
data = runs_per_match.merge(wickets_per_match,on="match_id")
```

```
data = data.merge(extras_per_match,on="match_id")
```

```
data.columns = ["match_id","runs","wickets","extras"]
```

```
print(data.head())
```

Step 4: Feature Selection

```
X = data[["runs","wickets","extras"]]
```

Step 5: Data Normalization

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

Step 6: Apply K-Means Clustering

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
```

```
clusters = kmeans.fit_predict(X_scaled)
```

```
data["cluster"] = clusters
```

```
print(data.head())
```

Step 7: Visualization

```
plt.figure(figsize=(8,6))
```

```
sns.scatterplot(  
    x=data["runs"],  
    y=data["wickets"],  
    hue=data["cluster"],  
    palette="Set1"  
)
```

```
plt.title("IPL Match Clustering based on Runs and Wickets")
```

```
plt.xlabel("Total Runs")
```

```
plt.ylabel("Wickets Lost")
```

```
plt.show()
```